Lesson 9 Teacher's Key: On the Scoreboard

TIC-80 - Instructor Reference Only

VIDEO GAME CREATION

What This Lesson Teaches

That TIC-80's print() has been the 'real' HUD tool the whole time - this lesson just moves it from conditional (You Win!/Game Over) to unconditional (always-visible score/lives), the same underlying mechanism used differently.

Level 1 - Full Worked Solution

- score=0 is a new global, parallel to lives and enemies_remaining.
- The score increment lives inside the bullets loop's hit branch - the exact same branch that already changes enemies_remaining, since both are consequences of the exact same confirmed hit.
- The two new print() calls at the bottom of TIC() run every single frame with no surrounding if-check, unlike the existing conditional You Win!/Game Over prints.

Highlighted lines are new this week; everything else is code you already have.

```
-- top of your code, alongside lives and enemies_remaining:
score=0

-- bullets loop's hit branch:
-- ...existing overlap check...
e.alive=false
enemies_remaining=enemies_remaining-1
score=score+10

-- bottom of TIC(), unconditional every frame:
print("Score: "..score,5,5,12)
print("Lives: "..lives,5,15,12)
```

Why This Structure

- print() was already the correct, idiomatic TIC-80 tool for on-screen text as far back as Lesson 6 - nothing new is being introduced, only a new (unconditional) way of calling something already familiar.
- Because these calls re-read score and lives directly every frame, the display can never fall out of sync - the same guarantee GameMaker's Draw GUI approach offers, for the same underlying reason (direct read, every frame, no caching).

Level 2 / Level 3 Talking Points

- Ask students to contrast the always-run score/lives prints with the conditional You Win!/Game Over prints in the same TIC() function - both use print(), the only difference is whether an if surrounds the call.
- For students working with larger carts: this is a good moment to ask whether TIC-80's single long TIC() function is starting to feel unwieldy, and whether breaking pieces into named functions (draw_hud(), update_enemies(), etc.) might help - a preview of concerns that grow through Lesson 11.

Level 2 - complete code:

```
-- Level 2 is verification only - the Level 1 code is already correct, so
no lines change.
-- Checks: 3 kills read exactly "Score: 30"; an escape moves lives but
never score;
-- both numbers are correct the instant the game starts, not only after
the first kill or escape.
```

Level 3 - complete code (all goals combined):

Highlighted lines are the Level 3 additions on top of the Level 1 code.

```
-- bullets loop's hit branch - Level 3:
e.alive=false
enemies_remaining=enemies_remaining-1
score=score+math.random(5,15)

-- bottom of TIC() - Level 3 (color is the last argument):
print("Score: "..score,5,5,14)
print("Lives: "..lives,5,15,6)
```

Common Student Mistakes

- Placing the new print() calls above TIC() (in the one-time setup code) instead of inside it - they'd only ever draw once, not every frame.
- Adding score=score+10 to the enemy-drawing loop's escape branch instead of the bullets loop's hit branch (makes escaping award points, which is backwards).
- Y-coordinate collisions between the new score/lives prints and the existing You Win!/Game Over prints.